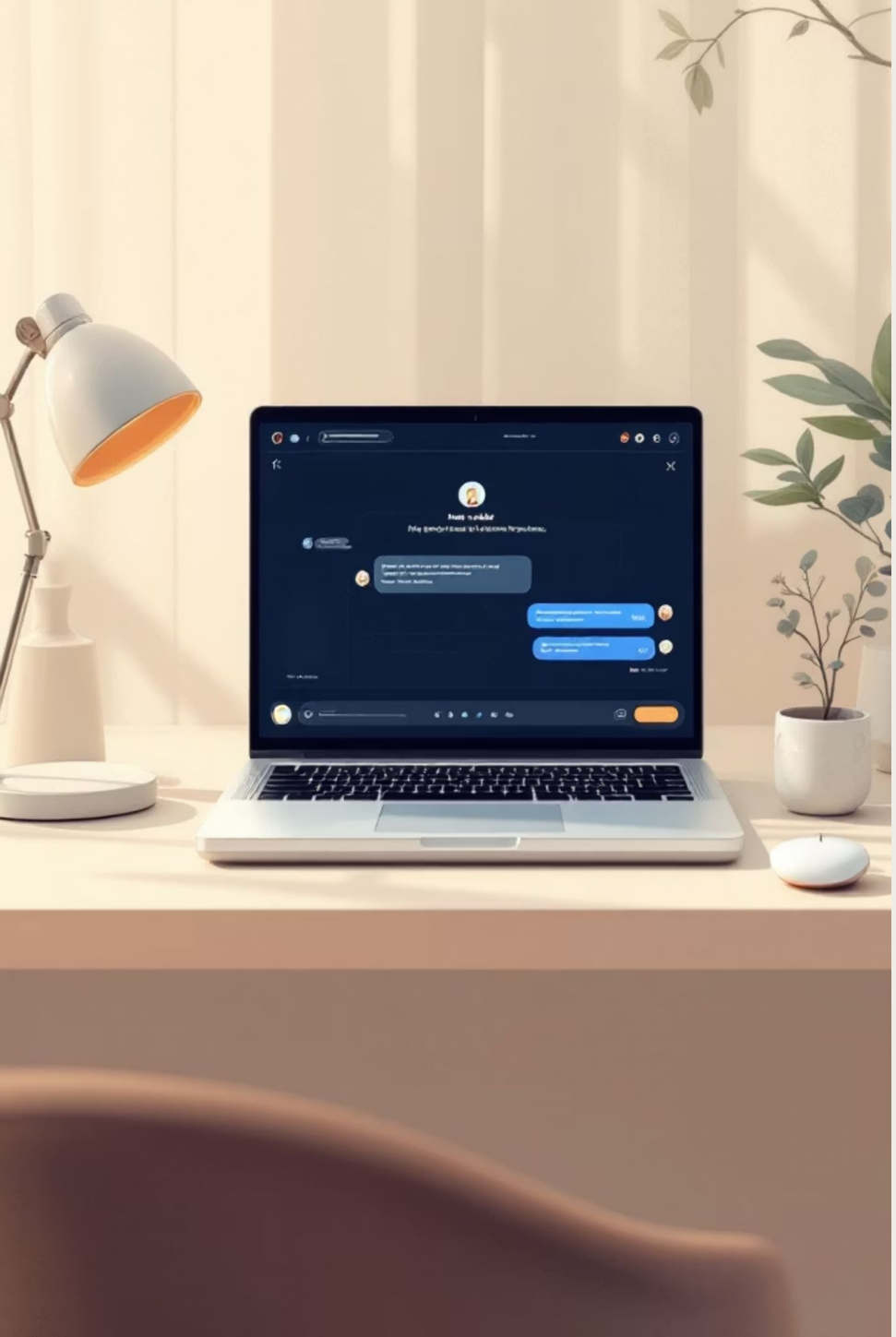

DSA AI Coder

Aplicação de IA com Python e Streamlit

Interface com Streamlit

Interface construída com Streamlit para capturar perguntas do usuário e exibir respostas em tempo real.

```
st.title("DSA AI Coder")
st.caption("Faça sua pergunta sobre Python")
prompt = st.chat_input("Qual sua dúvida sobre Python?")
```

Prompt Engineering

Definição de prompt estruturado para controlar o comportamento, formato e especialização do modelo de linguagem.

```
CUSTOM_PROMPT = """
Você é um assistente especialista em Python.

REGRAS:
- Responder apenas sobre programação
- Explicar de forma clara
- Gerar código com exemplos
"""
```

```
# Define um prompt de sistema que descreve as regras e comportamento do assistente de IA
CUSTOM_PROMPT = """
Você é o "DSA Coder", um assistente de IA especialista em programação, com foco principal
em Python. Sua missão é ajudar desenvolvedores iniciantes com dúvidas de programação de
forma clara, precisa e útil.

REGRAS DE OPERAÇÃO:
1.  **Foco em Programação**: Responda apenas a perguntas relacionadas a programação,
algoritmos, estruturas de dados, bibliotecas e frameworks. Se o usuário perguntar
sobre outro assunto, responda educadamente que seu foco é exclusivamente em auxiliar
com código.
2.  **Estrutura da Resposta**: Sempre formate suas respostas da seguinte maneira:
    * **Explicação Clara**: Comece com uma explicação conceitual sobre o tópico
    perguntado. Seja direto e didático.
    * **Exemplo de Código**: Forneça um ou mais blocos de código em Python com a
    sintaxe correta. O código deve ser bem comentado para explicar as partes
    importantes.
    * **Detalhes do Código**: Após o bloco de código, descreva em detalhes o que
     cada parte do código faz, explicando a lógica e as funções utilizadas.
    * **Documentação de Referência**: Ao final, inclua uma seção chamada "
    Documentação de Referência" com um link direto e relevante para a documentação
    oficial da Linguagem Python (docs.python.org) ou da biblioteca em questão.
3.  **Clareza e Precisão**: Use uma linguagem clara. Evite jargões desnecessários.
 Suas respostas devem ser tecnicamente precisas.
"""
```

Estrutura de Mensagens (MEMÓRIA)

Gerenciamento de contexto utilizando estrutura de mensagens para manter histórico da conversa.

```
if "messages" not in
st.session_state:
    st.session_state.messages = []

st.session_state.messages.append({
    "role": "user",
    "content": prompt
})

messages_for_api = [
 {"role": "system", "content":
CUSTOM_PROMPT}
]
```

```
for msg in st.session_state.messages:
 messages_for_api.append(msg)
```

```
# Inicializa o histórico de mensagens na sessão, caso ainda não exista
if "messages" not in st.session_state:
    st.session_state.messages = []

# Exibe todas as mensagens anteriores armazenadas no estado da sessão
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# Inicializa a variável do cliente Groq como None
client = None

# Verifica se o usuário forneceu a chave de API da Groq
if groq_api_key:

    try:

        # Cria cliente Groq com a chave de API fornecida
        client = Groq(api_key = groq_api_key)

    except Exception as e:

        # Exibe erro caso haja problema ao inicializar cliente
        st.sidebar.error(f"Erro ao inicializar o cliente Groq: {e}")
        st.stop()
```

```
# Caso não tenha chave, mas já existam mensagens, mostra aviso
elif st.session_state.messages:
     st.warning("Por favor, insira sua API Key da Groq na barra lateral para continuar.")

# Captura a entrada do usuário no chat
if prompt := st.chat_input("Qual sua dúvida sobre Python?"):

    # Se não houver cliente válido, mostra aviso e para a execução
    if not client:
        st.warning("Por favor, insira sua API Key da Groq na barra lateral para começar.")
        st.stop()

    # Armazena a mensagem do usuário no estado da sessão
    st.session_state.messages.append({"role": "user", "content": prompt})

    # Exibe a mensagem do usuário no chat
    with st.chat_message("user"):
        st.markdown(prompt)

    # Prepara mensagens para enviar à API, incluindo prompt de sistema
    messages_for_api = [{"role": "system", "content": CUSTOM_PROMPT}]
    for msg in st.session_state.messages:

        messages_for_api.append(msg)
```

Construção do Contexto - Integração com API

Integração com API de LLM utilizando parâmetros de controle como temperatura e limite de tokens.

```
chat_completion = client.chat.completions.create(
 messages=messages_for_api,
 model="openai/gpt-oss-20b",
 temperature=0.7,
 max_tokens=2048,
)
```

```
# Cria a resposta do assistente no chat
with st.chat_message("assistant"):

    with st.spinner("Analisando sua pergunta..."):

        try:

            # Chama a API da Groq para gerar a resposta do assistente
            chat_completion = client.chat.completions.create(
                messages = messages_for_api,
                model = "openai/gpt-oss-20b",
                temperature = 0.7,
                max_tokens = 2048,
            )

            # Extrai a resposta gerada pela API
            dsa_ai_resposta = chat_completion.choices[0].message.content

            # Exibe a resposta no Streamlit
            st.markdown(dsa_ai_resposta)

            # Armazena resposta do assistente no estado da sessão
            st.session_state.messages.append({"role": "assistant", "content": dsa_ai_resposta})

        # Caso ocorra erro na comunicação com a API, exibe mensagem de erro
        except Exception as e:
            st.error(f"Ocorreu um erro ao se comunicar com a API da Groq: {e}")

.markdown(
```

Processamento da Resposta

Processamento da resposta retornada (JSON) e exibição no frontend após extração do conteúdo.

Extração da resposta do modelo e renderização no frontend da aplicação.

```
dsa_ai_resposta =
chat_completion.choices[0].me
ssage.content
st.markdown(dsa_ai_resposta)
```

```python
# Cria a resposta do assistente no chat
with st.chat_message("assistant"):

    with st.spinner("Analisando sua pergunta..."):

        try:

            # Chama a API da Groq para gerar a resposta do assistente
            chat_completion = client.chat.completions.create(
                messages = messages_for_api,
                model = "openai/gpt-oss-20b",
                temperature = 0.7,
                max_tokens = 2048,
            )

            # Extrai a resposta gerada pela API
            dsa_ai_resposta = chat_completion.choices[0].message.content

            # Exibe a resposta no Streamlit
            st.markdown(dsa_ai_resposta)

            # Armazena resposta do assistente no estado da sessão
            st.session_state.messages.append({"role": "assistant", "content": dsa_ai_resposta})

        # Caso ocorra erro na comunicação com a API, exibe mensagem de erro
        except Exception as e:
            st.error(f"Ocorreu um erro ao se comunicar com a API da Groq: {e}")
```